

OpenCV সম্পূর্ণ গাইডলাইন - বেসিক থেকে অ্যাডভান্সড

সূচিপত্র

1. OpenCV পরিচিতি
 2. ইনস্টলেশন
 3. বেসিক কনসেপ্ট
 4. ইমেজ প্রসেসিং বেসিক
 5. ইমেজ ফিল্টারিং
 6. জিওমেট্রিক ট্রান্সফরমেশন
 7. কনট্যুর ডিটেকশন
 8. ফিচার ডিটেকশন
 9. ভিডিও প্রসেসিং
 10. মেশিন লার্নিং
 11. অ্যাডভান্সড টপিক
 12. প্রজেক্ট আইডিয়া
-

OpenCV পরিচিতি

OpenCV কি?

OpenCV (Open Source Computer Vision Library) হল একটি ওপেন সোর্স কম্পিউটার ভিশন, মেশিন লার্নিং এবং ইমেজ প্রসেসিং লাইব্রেরি। এটি Python, C++, Java সহ বিভিন্ন প্রোগ্রামিং ভাষায় ব্যবহার করা যায়।

OpenCV এর ব্যবহার:

- ইমেজ এবং ভিডিও প্রসেসিং
 - অবজেক্ট ডিটেকশন এবং রিকগনিশন
 - ফেস ডিটেকশন
 - মেশিন লার্নিং
 - অগমেন্টেড রিয়েলিটি
 - রোবটিক্স
-

ইনস্টলেশন

Python এ OpenCV ইনস্টল করা:

```
bash

# মূল OpenCV প্যাকেজ
pip install opencv-python

# অতিরিক্ত ফিচার সহ
pip install opencv-contrib-python

# NumPy (প্রয়োজনীয়)
pip install numpy

# Matplotlib (ভিজুয়ালাইজেশনের জন্য)
pip install matplotlib
```

প্রথম টেস্ট:

```
python

import cv2
import numpy as np

# OpenCV ভার্সন চেক
print("OpenCV Version:", cv2.__version__)

# একটি সাদা ইমেজ তৈরি
img = np.ones((300, 400, 3), dtype=np.uint8) * 255
cv2.imshow('Test Window', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

বেসিক কনসেপ্ট

ইমেজ কি?

ডিজিটাল ইমেজ হল পিক্সেলের একটি ম্যাট্রিক্স। প্রতিটি পিক্সেলে রং এর তথ্য থাকে।

রং মডেল:

- **Grayscale:** 0-255 (কালো থেকে সাদা)
- **RGB:** Red, Green, Blue

- **BGR**: Blue, Green, Red (OpenCV এর ডিফল্ট)
- **HSV**: Hue, Saturation, Value

কো-অর্ডিনেট সিস্টেম:

(0,0) ----- X
|
|
|
Y

ইমেজ প্রসেসিং বেসিক

ইমেজ পড়া, দেখানো এবং সেভ করা:

```
python

import cv2

# ইমেজ পড়া
img = cv2.imread('image.jpg') # রঙিন ইমেজ
img_gray = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE) # ধূসর ইমেজ

# ইমেজ দেখানো
cv2.imshow('Original Image', img)
cv2.imshow('Grayscale Image', img_gray)

# কী প্রেস পর্যন্ত অপেক্ষা
cv2.waitKey(0)
cv2.destroyAllWindows()

# ইমেজ সেভ করা
cv2.imwrite('saved_image.jpg', img)
```

ইমেজের বেসিক প্রোপার্টি:

```
python
```

```
import cv2

img = cv2.imread('image.jpg')

# ইমেজের আকার
print("Shape:", img.shape) # (height, width, channels)
print("Height:", img.shape[0])
print("Width:", img.shape[1])
print("Channels:", img.shape[2])

# ইমেজের ডেটা টাইপ
print("Data Type:", img.dtype)

# টোটাল পিক্সেল
print("Total Pixels:", img.size)
```

রং স্পেস কনভার্সন:

```
python

import cv2

img = cv2.imread('image.jpg')

# BGR থেকে RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# BGR থেকে Grayscale
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# BGR থেকে HSV
img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# দেখানো
cv2.imshow('Original', img)
cv2.imshow('Grayscale', img_gray)
cv2.imshow('HSV', img_hsv)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

ইমেজ ROI (Region of Interest):

```
python
```

```
import cv2
```

```
img = cv2.imread('image.jpg')
```

```
# নির্দিষ্ট অংশ কাটা
```

```
roi = img[100:300, 150:400] # [y1:y2, x1:x2]
```

```
cv2.imshow('ROI', roi)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

ইমেজ চ্যানেল স্প্লিট এবং মার্জ:

```
python
```

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread('image.jpg')
```

```
# চ্যানেল আলাদা করা
```

```
b, g, r = cv2.split(img)
```

```
# শুধু নীল চ্যানেল দেখানো
```

```
zeros = np.zeros(img.shape[:2], dtype="uint8")
```

```
blue_only = cv2.merge([b, zeros, zeros])
```

```
# চ্যানেল আবার যুক্ত করা
```

```
merged = cv2.merge([b, g, r])
```

```
cv2.imshow('Blue Channel', blue_only)
```

```
cv2.imshow('Merged', merged)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

ইমেজ ফিল্টারিং

নয়েজ রিমুভাল:

```
python
```

```
import cv2

img = cv2.imread('noisy_image.jpg')

# Gaussian Blur
gaussian_blur = cv2.GaussianBlur(img, (15, 15), 0)

# Median Filter
median_blur = cv2.medianBlur(img, 15)

# Bilateral Filter (এজ সংরক্ষণ করে)
bilateral = cv2.bilateralFilter(img, 9, 75, 75)

cv2.imshow('Original', img)
cv2.imshow('Gaussian Blur', gaussian_blur)
cv2.imshow('Median Blur', median_blur)
cv2.imshow('Bilateral Filter', bilateral)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

এজ ডিটেকশন:

```
python
```

```
import cv2
import numpy as np

img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Canny Edge Detection
edges = cv2.Canny(gray, 50, 150)

# Sobel Edge Detection
sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=5)
sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=5)

cv2.imshow('Original', img)
cv2.imshow('Canny Edges', edges)
cv2.imshow('Sobel X', sobelx)
cv2.imshow('Sobel Y', sobely)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

মরফোলজিক্যাল অপারেশন:

python

```
import cv2
import numpy as np

img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# বাইনারি ইমেজ তৈরি
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# কার্নেল তৈরি
kernel = np.ones((5,5), np.uint8)

# Erosion
erosion = cv2.erode(binary, kernel, iterations=1)

# Dilation
dilation = cv2.dilate(binary, kernel, iterations=1)

# Opening (erosion followed by dilation)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)

# Closing (dilation followed by erosion)
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

cv2.imshow('Original', binary)
cv2.imshow('Erosion', erosion)
cv2.imshow('Dilation', dilation)
cv2.imshow('Opening', opening)
cv2.imshow('Closing', closing)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

জিওমেট্রিক ট্রান্সফরমেশন

ইমেজ রিসাইজ করা:

```
python
```

```
import cv2

img = cv2.imread('image.jpg')

# নির্দিষ্ট সাইজে রিসাইজ
resized = cv2.resize(img, (300, 200))

# স্কেল ফ্যাক্টর দিয়ে রিসাইজ
scaled = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_CUBIC)

cv2.imshow('Original', img)
cv2.imshow('Resized', resized)
cv2.imshow('Scaled', scaled)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

ইমেজ ঘুরানো:

```
python

import cv2
import numpy as np

img = cv2.imread('image.jpg')
(h, w) = img.shape[:2]

# কেন্দ্রবিন্দু নির্ধারণ
center = (w // 2, h // 2)

# রোটেশন ম্যাট্রিক্স
M = cv2.getRotationMatrix2D(center, 45, 1.0)

# ইমেজ ঘুরানো
rotated = cv2.warpAffine(img, M, (w, h))

cv2.imshow('Original', img)
cv2.imshow('Rotated', rotated)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

ইমেজ ট্রান্সলেশন:

```
python
```

```
import cv2
import numpy as np

img = cv2.imread('image.jpg')
rows, cols = img.shape[:2]

# ট্রান্সলেশন ম্যাট্রিক্স
M = np.float32([[1, 0, 100], [0, 1, 50]]) # x=100, y=50 সরানো

# ইমেজ সরানো
translated = cv2.warpAffine(img, M, (cols, rows))

cv2.imshow('Original', img)
cv2.imshow('Translated', translated)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

পার্সপেক্টিভ ট্রান্সফরমেশন:

```
python

import cv2
import numpy as np

img = cv2.imread('image.jpg')

# চারটি পয়েন্ট নির্ধারণ
pts1 = np.float32([[56,65],[368,52],[28,387],[389,390]])
pts2 = np.float32([[0,0],[300,0],[0,300],[300,300]])

# পার্সপেক্টিভ ম্যাট্রিক্স
M = cv2.getPerspectiveTransform(pts1, pts2)

# পার্সপেক্টিভ ট্রান্সফরমেশন
dst = cv2.warpPerspective(img, M, (300, 300))

cv2.imshow('Original', img)
cv2.imshow('Perspective', dst)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

কনট্যুর ডিটেকশন

কনট্যুর খোঁজা এবং আঁকা:

```
python

import cv2
import numpy as np

# একটি বাইনারি ইমেজ তৈরি
img = np.zeros((400, 400, 3), dtype=np.uint8)
cv2.rectangle(img, (50, 50), (150, 150), (255, 255, 255), -1)
cv2.circle(img, (300, 300), 50, (255, 255, 255), -1)

# গ্রেস্কেল এবং থ্রেশহোল্ড
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# কনট্যুর খোঁজা
contours, hierarchy = cv2.findContours(gray, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# কনট্যুর আঁকা
result = img.copy()
cv2.drawContours(result, contours, -1, (0, 255, 0), 2)

print(f"Found {len(contours)} contours")

cv2.imshow('Original', img)
cv2.imshow('Contours', result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

কনট্যুরের প্রোপার্টি:

```
python
```

```

import cv2
import numpy as np

img = cv2.imread('shapes.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

for i, contour in enumerate(contours):
    # এরিয়া
    area = cv2.contourArea(contour)

    # পেরিমিটার
    perimeter = cv2.arcLength(contour, True)

    # বাউন্ডিং রেক্ট্যাঙ্গেল
    x, y, w, h = cv2.boundingRect(contour)

    # সেন্ট্রয়েড
    M = cv2.moments(contour)
    if M["m00"] != 0:
        cx = int(M["m10"] / M["m00"])
        cy = int(M["m01"] / M["m00"])

    print(f"Contour {i}: Area={area}, Perimeter={perimeter}")

    # বাউন্ডিং রেক্ট্যাঙ্গেল আঁকা
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)

    # সেন্ট্রয়েড মার্ক করা
    cv2.circle(img, (cx, cy), 5, (255, 0, 0), -1)

cv2.imshow('Result', img)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

শেপ ডিটেকশন:

python

```

import cv2
import numpy as np

def detect_shape(contour):
    # অ্যাপ্রক্সিমেট করা
    epsilon = 0.02 * cv2.arcLength(contour, True)
    approx = cv2.approxPolyDP(contour, epsilon, True)

    vertices = len(approx)

    if vertices == 3:
        return "Triangle"
    elif vertices == 4:
        # আয়তক্ষেত্র বা বর্গক্ষেত্র চেক
        (x, y, w, h) = cv2.boundingRect(approx)
        ar = w / float(h)
        shape = "Square" if ar >= 0.95 and ar <= 1.05 else "Rectangle"
        return shape
    elif vertices == 5:
        return "Pentagon"
    else:
        return "Circle"

img = cv2.imread('shapes.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

for contour in contours:
    if cv2.contourArea(contour) > 1000: # ছোট নয়েজ এড়ানো
        shape = detect_shape(contour)

        # বাউন্ডিং রেক্ট
        x, y, w, h = cv2.boundingRect(contour)

        # শেপের নাম লেখা
        cv2.putText(img, shape, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
        cv2.drawContours(img, [contour], -1, (0, 255, 0), 2)

cv2.imshow('Shape Detection', img)

```

```
cv2.waitKey(0)
cv2.destroyAllWindows()
```

ফিচার ডিটেকশন

কর্নার ডিটেকশন (Harris Corner):

```
python

import cv2
import numpy as np

img = cv2.imread('chessboard.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Harris Corner Detection
gray = np.float32(gray)
dst = cv2.cornerHarris(gray, 2, 3, 0.04)

# থ্রেশহোল্ড
dst = cv2.dilate(dst, None)
img[dst > 0.01 * dst.max()] = [0, 0, 255]

cv2.imshow('Harris Corners', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

SIFT ফিচার ডিটেকশন:

```
python
```

```
import cv2
```

```
img = cv2.imread('image.jpg')
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# SIFT ডিটেক্টর তৈরি
```

```
sift = cv2.SIFT_create()
```

```
# কী-পয়েন্ট এবং ডেস্ক্রিপ্টর খোঁজা
```

```
keypoints, descriptors = sift.detectAndCompute(gray, None)
```

```
# কী-পয়েন্ট আঁকা
```

```
img_with_keypoints = cv2.drawKeypoints(img, keypoints, None, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
```

```
print(f"Found {len(keypoints)} keypoints")
```

```
cv2.imshow('SIFT Keypoints', img_with_keypoints)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

ফিচার ম্যাচিং:

```
python
```

```
import cv2
```

```
# দুটি ইমেজ লোড করা
```

```
img1 = cv2.imread('image1.jpg', cv2.IMREAD_GRAYSCALE)
```

```
img2 = cv2.imread('image2.jpg', cv2.IMREAD_GRAYSCALE)
```

```
# SIFT ডিটেক্টর
```

```
sift = cv2.SIFT_create()
```

```
# কী-পয়েন্ট এবং ডেস্ক্রিপ্টর
```

```
kp1, des1 = sift.detectAndCompute(img1, None)
```

```
kp2, des2 = sift.detectAndCompute(img2, None)
```

```
# FLANN ম্যাচার
```

```
FLANN_INDEX_KDTREE = 1
```

```
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
```

```
search_params = dict(checks=50)
```

```
flann = cv2.FlannBasedMatcher(index_params, search_params)
```

```
matches = flann.knnMatch(des1, des2, k=2)
```

```
# ভাল ম্যাচ ফিল্টার করা
```

```
good_matches = []
```

```
for m, n in matches:
```

```
    if m.distance < 0.7 * n.distance:
```

```
        good_matches.append(m)
```

```
# ম্যাচ আঁকা
```

```
result = cv2.drawMatches(img1, kp1, img2, kp2, good_matches, None, flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

```
cv2.imshow('Feature Matching', result)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

ভিডিও প্রসেসিং

ওয়েবক্যাম থেকে ভিডিও:

```
python
```

```
import cv2

# ওয়েবক্যাম শুরু
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    # ফ্রেম প্রসেস করা (এখানে গ্রেস্কেল)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # দেখানো
    cv2.imshow('Webcam', frame)
    cv2.imshow('Gray', gray)

    # 'q' চাপলে বের হওয়া
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

ভিডিও ফাইল প্রসেসিং:

```
python
```

```

import cv2

# ভিডিও ফাইল লোড
cap = cv2.VideoCapture('video.mp4')

# ভিডিও প্রোপার্টি
fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

print(f"FPS: {fps}, Width: {width}, Height: {height}")

# আউটপুট ভিডিও রাইটার
fourcc = cv2.VideoWriter_fourcc(*'XVID')
out = cv2.VideoWriter('output.avi', fourcc, fps, (width, height))

while True:
    ret, frame = cap.read()

    if not ret:
        break

    # ফ্রেম প্রসেস করা (এজ ডিটেকশন)
    edges = cv2.Canny(frame, 50, 150)
    edges_colored = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

    # সেভ করা
    out.write(edges_colored)

    # দেখানো
    cv2.imshow('Original', frame)
    cv2.imshow('Edges', edges)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
out.release()
cv2.destroyAllWindows()

```

মোশন ডিটেকশন:

python

```

import cv2
import numpy as np

cap = cv2.VideoCapture(0)

# প্রথম ফ্রেম ব্যাকগ্রাউন্ড হিসেবে
ret, background = cap.read()
background = cv2.cvtColor(background, cv2.COLOR_BGR2GRAY)
background = cv2.GaussianBlur(background, (21, 21), 0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # কারেন্ট ফ্রেম প্রসেস
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray = cv2.GaussianBlur(gray, (21, 21), 0)

    # ব্যাকগ্রাউন্ড সাবট্রাকশন
    diff = cv2.absdiff(background, gray)
    thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)[1]

    # মরফোলজিক্যাল অপারেশন
    thresh = cv2.dilate(thresh, None, iterations=2)

    # কন্টুর খোঁজা
    contours, _ = cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    # মোশন ডিটেক্ট করা
    for contour in contours:
        if cv2.contourArea(contour) < 1000:
            continue

        (x, y, w, h) = cv2.boundingRect(contour)
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(frame, "Motion Detected", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

    cv2.imshow('Motion Detection', frame)
    cv2.imshow('Thresh', thresh)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

```

```
cap.release()  
cv2.destroyAllWindows()
```

মেশিন লার্নিং

ফেস ডিটেকশন (Haar Cascades):

```
python
```

```

import cv2

# হার ক্যাসকেড লোড করা
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # ফেস ডিটেক্ট করা
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    for (x, y, w, h) in faces:
        # ফেস বক্স আঁকা
        cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 0, 0), 2)

        # চোখের এলাকা
        roi_gray = gray[y:y+h, x:x+w]
        roi_color = frame[y:y+h, x:x+w]

        # চোখ ডিটেক্ট করা
        eyes = eye_cascade.detectMultiScale(roi_gray)
        for (ex, ey, ew, eh) in eyes:
            cv2.rectangle(roi_color, (ex, ey), (ex+ew, ey+eh), (0, 255, 0), 2)

    cv2.imshow('Face Detection', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

কালার ডিটেকশন এবং ট্র্যাকিং:

python

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # HSV কালার স্পেসে কনভার্ট
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # নীল রঙের রেঞ্জ (HSV তে)
    lower_blue = np.array([100, 50, 50])
    upper_blue = np.array([130, 255, 255])

    # নীল রঙের মাস্ক তৈরি
    mask = cv2.inRange(hsv, lower_blue, upper_blue)

    # নয়েজ কমানো
    mask = cv2.erode(mask, None, iterations=2)
    mask = cv2.dilate(mask, None, iterations=2)

    # কন্টুর খোঁজা
    contours, _ = cv2.findContours(mask.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    if len(contours) > 0:
        # সবচেয়ে বড় কন্টুর
        largest_contour = max(contours, key=cv2.contourArea)

        # সেন্ট্রয়েড খোঁজা
        M = cv2.moments(largest_contour)
        if M["m00"] != 0:
            cx = int(M["m10"] / M["m00"])
            cy = int(M["m01"] / M["m00"])

            # সেন্ট্রয়েড মার্ক করা
            cv2.circle(frame, (cx, cy), 10, (255, 0, 0), -1)
            cv2.putText(frame, f"Blue Object: ({cx}, {cy})", (cx-100, cy-20),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2)

    cv2.imshow('Original', frame)
```

```
cv2.imshow('Mask', mask)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

টেমপ্লেট ম্যাচিং:

```
python

import cv2
import numpy as np

# মূল ইমেজ এবং টেমপ্লেট লোড
img = cv2.imread('main_image.jpg')
template = cv2.imread('template.jpg')

# গ্রেস্কেলে কনভার্ট
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
template_gray = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)

# টেমপ্লেটের আকার
w, h = template_gray.shape[::-1]

# টেমপ্লেট ম্যাচিং
res = cv2.matchTemplate(img_gray, template_gray, cv2.TM_CCOEFF_NORMED)

# সর্বোচ্চ ম্যাচ খোঁজা
min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(res)

# থ্রেশহোল্ড সেট করা
threshold = 0.8

# সকল ভাল ম্যাচ খোঁজা
locations = np.where(res >= threshold)
for pt in zip(*locations[::-1]):
    cv2.rectangle(img, pt, (pt[0] + w, pt[1] + h), (0, 255, 0), 2)

cv2.imshow('Template Matching', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

অবজেক্ট ট্র্যাকিং:

python

```
import cv2

cap = cv2.VideoCapture(0)

# প্রথম ফ্রেম পড়া
ret, frame = cap.read()
if not ret:
    exit()

# ট্র্যাকিং এর জন্য ROI সিলেক্ট করা
bbox = cv2.selectROI("Select Object", frame, False)
cv2.destroyWindow("Select Object")

# ট্র্যাকার তৈরি
tracker = cv2.TrackerCSRT_create()
tracker.init(frame, bbox)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # ট্র্যাকার আপডেট
    success, bbox = tracker.update(frame)

    if success:
        # বাউন্ডিং বক্স আঁকা
        (x, y, w, h) = [int(v) for v in bbox]
        cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)
        cv2.putText(frame, "Tracking", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)
    else:
        cv2.putText(frame, "Lost", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

    cv2.imshow('Object Tracking', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

অ্যাডভান্সড টপিক

ইমেজ সেগমেন্টেশন (Watershed):

python

```

import cv2
import numpy as np
from matplotlib import pyplot as plt

# ইমেজ লোড
img = cv2.imread('coins.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# নয়েজ রিমুভাল
ret, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# মরফোলজিক্যাল অপারেশন
kernel = np.ones((3,3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)

# সিওর ব্যাকগ্রাউন্ড এরিয়া
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# সিওর ফোরগ্রাউন্ড এরিয়া
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
ret, sure_fg = cv2.threshold(dist_transform, 0.7*dist_transform.max(), 255, 0)

# আননোন এরিয়া
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

# মার্কার লেবেলিং
ret, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

# ওয়াটারশেড
markers = cv2.watershed(img, markers)
img[markers == -1] = [255, 0, 0]

cv2.imshow('Watershed Segmentation', img)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

স্টেরিও ভিশন (Depth Map):

python

```
import cv2
import numpy as np

# স্টেরিও ইমেজ লোড
imgL = cv2.imread('left_image.jpg', cv2.IMREAD_GRAYSCALE)
imgR = cv2.imread('right_image.jpg', cv2.IMREAD_GRAYSCALE)

# স্টেরিও BM অবজেক্ট তৈরি
stereo = cv2.StereoBM_create(numDisparities=16, blockSize=15)

# ডিসপারিটি ম্যাপ কম্পিউট করা
disparity = stereo.compute(imgL, imgR)

# নরমালাইজ করা
disparity_normalized = cv2.normalize(disparity, None, alpha=0, beta=255,
                                     norm_type=cv2.NORM_MINMAX, dtype=cv2.CV_8U)

cv2.imshow('Left Image', imgL)
cv2.imshow('Right Image', imgR)
cv2.imshow('Disparity Map', disparity_normalized)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

অপটিক্যাল ফ্লো:

```
python
```

```

import cv2
import numpy as np

cap = cv2.VideoCapture(0)

# প্যারামিটার সেট করা
feature_params = dict(maxCorners=100, qualityLevel=0.3, minDistance=7, blockSize=7)
lk_params = dict(winSize=(15,15), maxLevel=2, criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03))

# র্যান্ডম কালার
colors = np.random.randint(0, 255, (100, 3))

# প্রথম ফ্রেম এবং পয়েন্ট
ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)

# ট্র্যাক তৈরির জন্য মাস্ক
mask = np.zeros_like(old_frame)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # অপটিক্যাল ফ্লো ক্যালকুলেট
    p1, st, err = cv2.calcOpticalFlowPyrLK(old_gray, frame_gray, p0, None, **lk_params)

    # ভাল পয়েন্ট সিলেক্ট
    if p1 is not None:
        good_new = p1[st==1]
        good_old = p0[st==1]

    # ট্র্যাক আঁকা
    for i, (new, old) in enumerate(zip(good_new, good_old)):
        a, b = new.ravel().astype(int)
        c, d = old.ravel().astype(int)
        mask = cv2.line(mask, (a, b), (c, d), colors[i].tolist(), 2)
        frame = cv2.circle(frame, (a, b), 5, colors[i].tolist(), -1)

    img = cv2.add(frame, mask)

```

```
cv2.imshow('Optical Flow', img)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# আপডেট
old_gray = frame_gray.copy()
p0 = good_new.reshape(-1, 1, 2)

cap.release()
cv2.destroyAllWindows()
```

ব্যাকগ্রাউন্ড সাবট্রাকশন:

```
python
```

```
import cv2

cap = cv2.VideoCapture(0)

# ব্যাকগ্রাউন্ড সাবট্রাক্টর তৈরি
backSub = cv2.createBackgroundSubtractorMOG2(detectShadows=True)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # ব্যাকগ্রাউন্ড সাবট্রাকশন
    fgMask = backSub.apply(frame)

    # মরফোলজিক্যাল অপারেশন
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    fgMask = cv2.morphologyEx(fgMask, cv2.MORPH_OPEN, kernel)

    # কন্টুর খোঁজা
    contours, _ = cv2.findContours(fgMask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    # বড় কন্টুর আঁকা
    for contour in contours:
        if cv2.contourArea(contour) > 500:
            x, y, w, h = cv2.boundingRect(contour)
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

    cv2.imshow('Original', frame)
    cv2.imshow('Foreground Mask', fgMask)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

প্রজেক্ট আইডিয়া

1. ডকুমেন্ট স্ক্যানার:

python

```

import cv2
import numpy as np

def order_points(pts):
    # পয়েন্ট অর্ডার করা
    rect = np.zeros((4, 2), dtype="float32")

    s = pts.sum(axis=1)
    rect[0] = pts[np.argmin(s)] # টপ-লেফট
    rect[2] = pts[np.argmax(s)] # বটম-রাইট

    diff = np.diff(pts, axis=1)
    rect[1] = pts[np.argmin(diff)] # টপ-রাইট
    rect[3] = pts[np.argmax(diff)] # বটম-লেফট

    return rect

def four_point_transform(image, pts):
    # পয়েন্ট অর্ডার করা
    rect = order_points(pts)
    (tl, tr, br, bl) = rect

    # নতুন ইমেজের width
    widthA = np.sqrt(((br[0] - bl[0]) ** 2) + ((br[1] - bl[1]) ** 2))
    widthB = np.sqrt(((tr[0] - tl[0]) ** 2) + ((tr[1] - tl[1]) ** 2))
    maxWidth = max(int(widthA), int(widthB))

    # নতুন ইমেজের height
    heightA = np.sqrt(((tr[0] - br[0]) ** 2) + ((tr[1] - br[1]) ** 2))
    heightB = np.sqrt(((tl[0] - bl[0]) ** 2) + ((tl[1] - bl[1]) ** 2))
    maxHeight = max(int(heightA), int(heightB))

    # গন্তব্য পয়েন্ট
    dst = np.array([
        [0, 0],
        [maxWidth - 1, 0],
        [maxWidth - 1, maxHeight - 1],
        [0, maxHeight - 1]], dtype="float32")

    # পার্সপেক্টিভ ট্রান্সফর্ম
    M = cv2.getPerspectiveTransform(rect, dst)
    warped = cv2.warpPerspective(image, M, (maxWidth, maxHeight))

```

return warped

def scan_document():

cap = cv2.VideoCapture(0)

while True:

ret, frame = cap.read()

if not ret:

break

প্রিপ্রসেসিং

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

gray = cv2.GaussianBlur(gray, (5, 5), 0)

edged = cv2.Canny(gray, 75, 200)

কন্টুর খোঁজা

contours, _ = cv2.findContours(edged.copy(), cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)

contours = sorted(contours, key=cv2.contourArea, reverse=True)[:5]

ডকুমেন্ট কন্টুর খোঁজা

for c in contours:

কন্টুর অ্যাপ্রক্সিমেট করা

peri = cv2.arcLength(c, True)

approx = cv2.approxPolyDP(c, 0.02 * peri, True)

যদি চারটি পয়েন্ট থাকে, তাহলে ডকুমেন্ট

if len(approx) == 4:

screenCnt = approx

break

else:

screenCnt = None

কন্টুর আঁকা

if screenCnt is not None:

cv2.drawContours(frame, [screenCnt], -1, (0, 255, 0), 2)

cv2.imshow('Document Scanner', frame)

cv2.imshow('Edged', edged)

key = cv2.waitKey(1) & 0xFF

if key == ord('q'):

break

elif key == ord('s') and screenCnt is not None:

স্ক্যান করা

```
warped = four_point_transform(frame, screenCnt.reshape(4, 2))  
cv2.imshow('Scanned', warped)
```

```
cap.release()  
cv2.destroyAllWindows()
```

```
# scan_document()
```

2. হ্যান্ড জেসচার রিকগনিশন:

```
python
```

```
import cv2
import numpy as np

def hand_gesture_recognition():
    cap = cv2.VideoCapture(0)

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        # HSV কনভারশন
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # স্কিনের রং রেঞ্জ
        lower_skin = np.array([0, 20, 70], dtype=np.uint8)
        upper_skin = np.array([20, 255, 255], dtype=np.uint8)

        # স্কিন মাস্ক
        mask = cv2.inRange(hsv, lower_skin, upper_skin)

        # নয়েজ রিমুভাল
        mask = cv2.medianBlur(mask, 19)

        # কনট্যুর খোঁজো
        contours, _ = cv2.findContours(mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

        if contours:
            # সবচেয়ে বড় কনট্যুর (হাত)
            max_contour = max(contours, key=cv2.contourArea)

            if cv2.contourArea(max_contour) > 10000:
                # কনভেক্স হাল
                hull = cv2.convexHull(max_contour, returnPoints=False)
                defects = cv2.convexityDefects(max_contour, hull)

                # ফিঙ্গার কাউন্ট
                finger_count = 0
                if defects is not None:
                    for i in range(defects.shape[0]):
                        s, e, f, d = defects[i, 0]
                        start = tuple(max_contour[s][0])
                        end = tuple(max_contour[e][0])
```

```

far = tuple(max_contour[f][0])

# ত্রিভুজের দূরত্ব
a = np.sqrt((end[0] - start[0])**2 + (end[1] - start[1])**2)
b = np.sqrt((far[0] - start[0])**2 + (far[1] - start[1])**2)
c = np.sqrt((end[0] - far[0])**2 + (end[1] - far[1])**2)

# কোসাইন নিয়ম
angle = np.arccos((b**2 + c**2 - a**2) / (2*b*c))

if angle <= np.pi/2:
    finger_count += 1
    cv2.circle(frame, far, 8, [211, 84, 0], -1)

# ফলাফল দেখানো
cv2.putText(frame, f'Fingers: {finger_count}', (50, 50),
            cv2.FONT_HERSHEY_SIMPLEX, 2, (255, 0, 0), 3)

# কনট্যুর আঁকা
cv2.drawContours(frame, [max_contour], -1, (0, 255, 0), 2)

cv2.imshow('Hand Gesture', frame)
cv2.imshow('Mask', mask)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

# hand_gesture_recognition()

```

3. লাইন ফলোয়িং রোবট সিমুলেশন:

python

```

import cv2
import numpy as np

def line_following_simulation():
    # একটি ট্র্যাক তৈরি করা
    track = np.zeros((400, 600, 3), dtype=np.uint8)

    # লাইন আঁকা
    cv2.line(track, (50, 200), (150, 100), (255, 255, 255), 10)
    cv2.line(track, (150, 100), (300, 100), (255, 255, 255), 10)
    cv2.line(track, (300, 100), (450, 200), (255, 255, 255), 10)
    cv2.line(track, (450, 200), (550, 300), (255, 255, 255), 10)

    # রোবট পজিশন
    robot_x, robot_y = 50, 200

    while True:
        current_track = track.copy()

        # রোবট আঁকা
        cv2.circle(current_track, (robot_x, robot_y), 15, (0, 0, 255), -1)

        # সেন্সর এরিয়া (রোবটের সামনে)
        sensor_region = current_track[robot_y-30:robot_y+30, robot_x:robot_x+60]

        if sensor_region.size > 0:
            # গ্রেস্কেল কনভার্ট
            gray_sensor = cv2.cvtColor(sensor_region, cv2.COLOR_BGR2GRAY)

            # সাদা পিক্সেল খোঁজা
            white_pixels = np.where(gray_sensor > 200)

            if len(white_pixels[0]) > 0:
                # লাইনের সেন্টার খোঁজা
                center_y = np.mean(white_pixels[0])
                center_x = np.mean(white_pixels[1])

                # রোবট মুভমেন্ট
                if center_y < 20: # লাইন উপরে
                    robot_y -= 2
                elif center_y > 40: # লাইন নিচে
                    robot_y += 2

```

```
robot_x += 2 # সামনের দিকে এগিয়ে যাওয়া
```

```
# সীমানার মধ্যে রাখা
```

```
robot_x = max(15, min(robot_x, 585))
```

```
robot_y = max(15, min(robot_y, 385))
```

```
# রিস্টার্ট
```

```
if robot_x > 580:
```

```
    robot_x, robot_y = 50, 200
```

```
cv2.imshow('Line Following Robot', current_track)
```

```
if cv2.waitKey(30) & 0xFF == ord('q'):
```

```
    break
```

```
cv2.destroyAllWindows()
```

```
# line_following_simulation()
```

4. QR কোড ডিটেক্টর:

```
python
```

```
import cv2
import numpy as np

def qr_code_detector():
    cap = cv2.VideoCapture(0)

    # QR কোড ডিটেক্টর
    qr_detector = cv2.QRCodeDetector()

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        # QR কোড ডিটেক্ট এবং ডিকোড
        data, bbox, rectified_qr = qr_detector.detectAndDecode(frame)

        if bbox is not None:
            # বাউন্ডিং বক্স আঁকা
            bbox = bbox.astype(int)
            cv2.polylines(frame, [bbox], True, (0, 255, 0), 2)

            if data:
                # QR কোডের ডেটা দেখানো
                cv2.putText(frame, f'QR Data: {data}', (10, 30),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

                # রেক্টিফাইড QR কোড দেখানো
                if rectified_qr is not None:
                    rectified_qr = cv2.resize(rectified_qr, (200, 200))
                    frame[10:210, frame.shape[1]-210:frame.shape[1]-10] = cv2.cvtColor(rectified_qr, cv2.COLOR_GRAY2BGR)

            cv2.imshow('QR Code Detector', frame)

            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

        cap.release()
        cv2.destroyAllWindows()

# qr_code_detector()
```

টিপস এবং ট্রিকস

পারফরম্যান্স অপ্টিমাইজেশন:

1. **ইমেজ সাইজ কমানো:** বড় ইমেজের জন্য প্রসেসিং টাইম বেশি
2. **ROI ব্যবহার:** শুধু প্রয়োজনীয় অংশ প্রসেস করা
3. **Numpy অপারেশন:** লুপের পরিবর্তে ভেক্টরাইজড অপারেশন
4. **মাল্টি-থ্রেডিং:** ভারী প্রসেসিংয়ের জন্য

সাধারণ সমস্যা এবং সমাধান:

python

```
# 1. ইমেজ না দেখানোর সমস্যা
cv2.imshow('Image', img)
cv2.waitKey(0) # এই লাইনটি গুরুত্বপূর্ণ
cv2.destroyAllWindows()

# 2. মেমোরি লিক এড়ানো
cap = cv2.VideoCapture(0)
# ... ব্যবহার করা
cap.release() # সবসময় রিলিজ করা

# 3. ইমেজ টাইপ চেক করা
if img is not None and img.size > 0:
    # প্রসেস করা
    pass

# 4. কালার চ্যানেল চেক
if len(img.shape) == 3: # রঙিন ইমেজ
    # RGB/BGR অপারেশন
    pass
elif len(img.shape) == 2: # গ্রেস্কেল
    # গ্রেস্কেল অপারেশন
    pass
```

রিসোর্স এবং রেফারেন্স

অফিসিয়াল ডকুমেন্টে